

Junior Makers · Class 29

Python Number Guessing Game

The computer thinks of a number. Can you beat it?

input & variables

if / elif / else

while loops

Strategy

Did you know?

With the best strategy you can find any number between 1 and 1,000,000 in just 20 guesses!

I'm thinking of 1-100...

Guess: 50

Too high!

Guess: 25

Too low!

Guess: 37

Correct in 3 guesses!



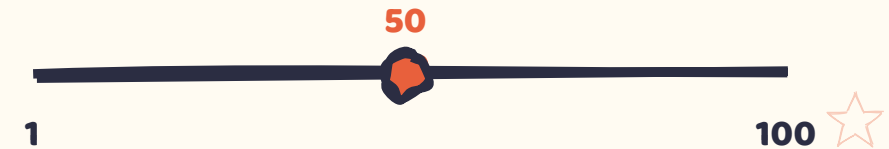
Let's think...

Can you guess my number in 7 tries?



1 to 100

I'm thinking of a number from 1 to 100. You can guess, and I'll say **too high** or **too low**. Most people need about 10 guesses. Is there a **trick** that always works in 7 or fewer?



Guess randomly?

Count up from 1?

There's a trick!

Today: you write a **guessing game in Python**, then discover the strategy computers use to search super fast.





Learning objectives

By the end of this class I can...



1

**Input & variables:** use `input()`, `int()` and **variables** to talk to the player**Proof:** my program asks my name and greets me

2

**Decisions:** use **if** / **elif** / **else** to say too high, too low or correct**Proof:** my game gives the right hint every time

3

**Loops:** use a **while loop** and **random** to make a full game with a guess counter**Proof:** my game counts how many guesses it took

4

**Strategy:** explain the **halving strategy** (binary search) and why it wins**Proof:** I can find any number 1–100 in 7 guesses or fewer



Class game

Human guessing game



How to play

1. Teacher thinks of a number from 1 to 100 and writes it hidden.
2. The class guesses. Teacher only says **too high**, **too low** or **correct**.
3. Count the guesses on the board.
4. Play again: this time a team of 3 decides each guess together.
5. Which round needed fewer guesses?

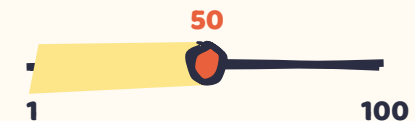
Then discuss

Did anyone use a smart strategy?
What was it?



Big idea

Every 'too high' or 'too low' throws away a chunk of numbers. Smart guessing throws away the biggest chunk.



Objective 4 · Strategy · video

Video time!



Binary Search

BBC Bitesize (Too Tall Productions) · about 2 minutes ·
youtu.be/8TNLRK_FyFc

Watch for...

- 1 Why do we look in the **middle** first?
- 2 What happens to the list after each guess?
- 3 Why does binary search need the list to be **in order**?



After the video: Think of a number from 1 to 100.
How many guesses would the best strategy need, at most?



Objective 1 · Input & variables

Talking to the player

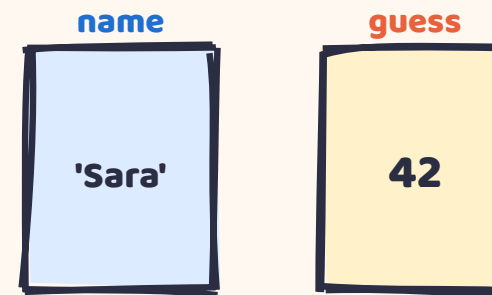


The idea

- `print("Hello")`: show text
- `name = input("Your name? ")`: ask and store the answer in a variable
- `input()` always gives **text**. To do maths, turn it into a number: `guess = int(input("Guess: "))`

"5" (text) and 5 (number) look the same but are different to Python!

Example: a labelled jar



A variable is like a jar with a label. 'name' holds 'Sara'. 'guess' holds 42. You can look inside, or swap what's inside, any time.



Objective 1 · Input & variables · Build step 1 of 4

Ask, store, show



```
name = input("What is your name? ")
print("Hello", name + "! Let's play.")

secret = 37                                # we'll make this random later
guess = int(input("Guess a number from 1 to 100: "))
print("You guessed", guess)
```



Thonny



Open **Thonny**, type the code, save as **guess.py**, click **Run**. Type your answers in the Shell at the bottom.

Try this

Type **hello** instead of a number. What error do you get? Why?

Check

The program greets you by name and repeats your guess.





Objective 2 · Decisions



Decisions with if, elif, else

The idea

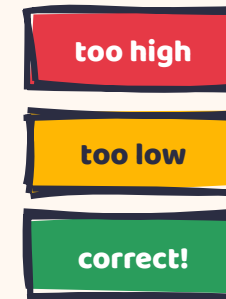
if guess > secret: → "Too high!"

elif guess < secret: → "Too low!"

else: → "Correct!"

Python checks them **in order** and runs only the **first** one that is true. **elif** means 'else if'. The indented lines (4 spaces) belong to each choice.

Example: a road junction sign



only **ONE** runs

At a junction: if you want Lahore, go left; else if you want Murree, go right; else go straight. You only take ONE road. if/elif/else picks exactly one path.



Objective 2 · Decisions · Build step 2 of 4

Give the right hint



```
secret = 37
guess = int(input("Guess a number from 1 to 100: "))

if guess > secret:
    print("Too high!")
elif guess < secret:
    print("Too low!")
else:
    print("Correct! Well done!")
```

Test all 3

Run it 3 times: guess 50, then 20, then 37. Do you see all three messages?

Colon & indent

Every if/elif/else line ends with : and the next line is indented 4 spaces.

== VS =

= stores a value. == asks 'are these equal?'

?



Objective 3 · Loops



Keep going with a while loop

The idea

One guess isn't a game. A **while loop** repeats **as long as** something is true:

while guess != secret: (keep going while the guess is NOT the secret)

import random and **random.randint(1, 100)** make the computer pick a different secret every game. A **counter** variable adds 1 each turn.

Example: 'keep stirring until it boils'



until correct

A recipe says: keep stirring while the milk is not boiling. You don't know how many stirs it will take. That's a while loop: repeat until the condition changes.



Objective 3 · Loops · Build step 3 of 4

The full game

```
import random

secret = random.randint(1, 100)
guesses = 0
guess = 0
print("I'm thinking of a number from 1 to 100...")

while guess != secret:
    guess = int(input("Your guess: "))
    guesses = guesses + 1
    if guess > secret:
        print("Too high!")
    elif guess < secret:
        print("Too low!")

print("Correct! It was", secret)
print("You took", guesses, "guesses.")
```

Why guess = 0?

The while line checks guess before the first input, so guess needs a starting value that can't be the secret.

Counter

`guesses = guesses + 1` adds one every loop.
That's how games count turns, points and lives.

Check

Play 3 games. Is the secret different each time?



Objective 4 · Strategy

The halving trick (binary search)



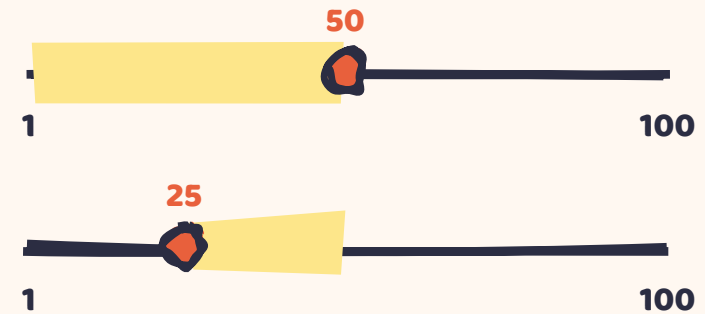
The idea

Always guess the **middle** of the numbers that are still possible:

1–100 → guess **50** → too high → now 1–49 → guess **25** → too low → now 26–49 → guess **37**...

Each guess throws away **half**. 100 → 50 → 25 → 13 → 7 → 4 → 2 → 1. That's at most **7 guesses**!

Example: finding a word in a dictionary



You don't read a dictionary from page 1. You open it in the middle, see if your word comes before or after, and throw away half. Search engines and computers use this trick billions of times a day.



Objective 4 · Strategy · test the trick

Random vs halving



Guesses (random)	Guesses (halving)
Game 1	
Game 2	
Game 3	
Average	



How to test

Play 3 games guessing however you like.
Then play 3 games using ONLY the halving
trick. Compare the averages.

Never more than 7
With halving, 1 to 100 never needs more than 7
guesses. Did you prove it?





Objective 3 · Loops · Build step 4 of 4

Polish your game



```
# add these to make your game better:

if guesses <= 7:
    print("Amazing! You used the halving trick!")
else:
    print("Try guessing the middle next time.")

again = input("Play again? (y/n) ")

# safety: what if the player types 'banana'?
text = input("Your guess: ")
if text.isdigit():
    guess = int(text)
else:
    print("Please type a number!")
```

Rating

Tell the player how well they did.

Play again

Put the whole game inside **while again == 'y':** to keep playing.

No crashes

.isdigit() checks the text is a number before **int()**, so 'banana' doesn't crash the game.





Real world

Binary search is everywhere



Search engines

Find pages among billions in a blink



Contacts

Your phone finds a name in a sorted list



Games

Computer players search for the best move



Fixing bugs

Programmers halve their code to find the broken line



How many guesses would halving need for 1 to 1,000? (Hint: keep halving 1,000.)

Answer

10 guesses! Doubling the range only adds ONE more guess.





Test like an engineer

Bug hunt



Bug	Likely cause	Did it happen?
ValueError: invalid literal for int()	player typed letters	
IndentationError	lines inside if/while not indented	
Loop never ends	guess not updated inside the loop	
Says 'Too high' for every guess	comparing text with a number (missing int)	
Same secret every game	secret = 37 still there	

Debug checklist

Add `print(secret)` while testing so you can check the hints. Delete it before others play!



Read errors from the bottom line up. Python tells you the line number.



Level up!

Coder challenges

Choose your challenge



Levels: Easy (1–10), Medium (1–100), Hard (1–1000).



Limited guesses: only 7 tries, then 'Game over!'.



Reverse game: YOU think of a number; the COMPUTER guesses using halving.



High score: remember the fewest guesses ever in a variable.

Think like an engineer

Why does the halving trick only work if the numbers are in order?

Boss level

Build the reverse game: the computer must never need more than 7 guesses.

?



Objective 3 · Loops · share

Swap and play



Play 3 friends' games

Can you break their game by typing something strange?
Did it give the right hints? Write one kind comment and one tip on a sticky note.

Tester card

Hints always correct?

Counts guesses?

Crashes on 'banana'?

Best feature





Objective check

Can I really do it? Prove it.



01 What does `input()` give you, and why do we need `int()`?



02 Show the `if / elif / else` that gives the hints.



03 Point to the `while` loop and the guess counter in your code.



04 Find my number from 1 to 100 using halving. How many guesses did you need?



Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Class tournament: fewest guesses over 3 games wins, using anyone's program.





Show what you know

What did we learn?

- ☒ `input()` reads text; `int()` turns it into a number
- ☒ `if / elif / else` picks exactly one path
- ☒ `while` repeats until the condition is false
- ☒ `random.randint` makes a new secret each game
- ☒ Halving finds any number from 1 to 100 in 7 guesses



At home

Play the guessing game with someone at home, using halving. Can they beat you without it?



Next class: Data Detectives: Class Survey Charts!



Exit ticket: how many guesses does halving need for 1 to 1,000?

